

ChatGPT

Nuova chat

Libreria

Progetti

Programmati

Plugin

Codex

Altro

Recenti

Stato progetto PBD

PBD 33 - Var rag/cont/uma T68 f...

Clausole risolutive versamenti

PBD 32 - Python PEP PyNav

PBD 31

DIRETTIVE DI COLLABORAZIONE...

PBD 30

PBD 28

Costo spedizione reso Polonia

PBD 29

Avviso accesso Telegram

PBD 27 - TPS 4

Pompa freno Alfa 147

Consulto riparazione coppa olio

RB

Riccardo Bendotti

Plus

- pbdd\_path(38).py**

Python
- pbdd\_types(31).py**

Python
- ui(136).py**

Python
- animation(24).py**

Python
- config(55).py**

Python
- const(63).py**

Python

Povegliano Veronese - 23/07/2026 10:43

# PBD - Stato progetto

## Stato attuale

È stata completata la rifattorizzazione del modulo `config`, compresa `Config.Advanced.load()`, ed è stato consolidato il nuovo modello di gestione degli errori. L'architettura dei percorsi dell'applicazione è ormai definita e costituisce la base per la distribuzione finale.

Mostra di più



Perfetto, contesto recuperato.

Dal materiale che mi hai passato emerge chiaramente una cosa che considero positiva: la rifattorizzazione di `config` non è stata soltanto un porting meccanico, ma è servita a stabilizzare il modello delle responsabilità.

La fotografia che mi porto avanti è questa.

### Config è ormai un modulo "infrastrutturale"

Il suo compito è limitato a:

- leggere e scrivere la configurazione;
- distinguere il caso "configurazione assente" (che è uno stato valido) da un vero errore;
- convertire eventuali errori strutturali (formato JSON, I/O ecc.) nelle eccezioni del nuovo modello;
- effettuare il backup/restore quando richiesto.

Non prende invece decisioni sul flusso applicativo. Questo si vede abbastanza chiaramente nei metodi `load()`, `save()`, `restore()` e anche in `Advanced.load()`: recuperano lo stato oppure propagano l'eccezione, senza decidere cosa fare dell'errore. `config`

### Dove vengono prese le decisioni

Le decisioni di recupero appartengono ai moduli chiamanti.

Un esempio molto pulito è `StorageDirs.init()`.

Lì si vede la separazione che avevamo discusso:

- `config.load()` può fallire;
- `StorageDirs` decide che, in questo specifico contesto, è possibile tentare un recupero;
- esegue `config.restore()`;
- riprova la lettura;
- informa l'utente del ripristino.

Questa è una decisione di business del modulo `storage`, non di `config`, ed è esattamente il tipo di architettura che volevamo ottenere. `pbdd_path`

### Conseguenza sul porting rimanente

Per questo motivo mi aspetto che il lavoro residuo su:

- `storage`
- `animation`

non consista tanto nello spostare `try/except`, quanto nel rispondere sistematicamente a una domanda:

questo modulo è in grado di recuperare l'errore?

Se la risposta è:

- sì, il modulo gestisce localmente l'errore;
- no, propaga la `PBDError`.

È una distinzione molto più architetturale che sintattica.

### Animation sarà probabilmente il banco di prova

Guardando l'attuale `MultiMediaManager` noto che è ancora nella fase "pre-porting": carica configurazioni e salva preferenze, ma i metodi operativi (`load_images()`, `build_frames()`, `to_gif()`, `to_webm()`) sono ancora degli stub. `animation`

Ed è proprio lì che il nuovo modello degli errori diventerà interessante.

Ad esempio, sarà necessario distinguere casi come:

- ZIP ugoira corrotto;
- frame mancante;
- immagine illeggibile;
- FFmpeg assente;
- FFmpeg terminato con errore;
- codec non supportato;
- impossibilità di scrivere il file finale.

Per ognuno di questi dovremo decidere:

- è recuperabile?
- va saltata solo quell'opera?
- va riprovata?
- oppure bisogna interrompere il processo?

Nel complesso direi che la rifattorizzazione sta seguendo una linea molto coerente:

1. i moduli di servizio (config, file, path...) producono dati o eccezioni;
2. i moduli di livello superiore decidono il comportamento;
3. i processi terminali (download, conversione, menu, ecc.) definiscono definitivamente la politica di recovery.

Questa è esattamente la separazione delle responsabilità che avevamo delineato, e rende il porting dei moduli rimanenti molto più lineare rispetto a quanto fosse all'inizio.

Ti piace questa personalità?

